

Forensic Image Detector

I suggest 2 approaches: first, fine-tuning an existing model that was trained on frequency masking with an RGB image. While the second approach will fine-tune on the existing model (Resnet50) with DCT map input. Both approaches share the same goal: build a binary classifier that detects AI-generated images using the GenImage dataset.

Dataset

- Source: GenImage.
 - Real images: ImageNet (included in GenImage as the nature class)
- Create three separate directory trees. They must never overlap.

genimage_test (Final evaluation only)

Source: val split from GenImage. One folder per generator. NEVER used during training or model selection.

```
genimage_test/
├─ midjourney/  ├─ 0_real/      └─ 1_fake/
├─ sdv1_4/      ├─ 0_real/      └─ 1_fake/
├─ sdv1_5/      ├─ 0_real/      └─ 1_fake/
├─ wukong/       ├─ 0_real/      └─ 1_fake/
├─ vqdm/         ├─ 0_real/      └─ 1_fake/
├─ biggan/       ├─ 0_real/      └─ 1_fake/
├─ adm/          ├─ 0_real/      └─ 1_fake/
└─ glide/        ├─ 0_real/      └─ 1_fake/
```

genimage_train (Training and validation monitoring)

All generators merged into a single flat structure.

```
genimage_train/
├─ train/
│   ├─ 0_real/  ← all real images, all generators, train split
│   └─ 1_fake/  ← all fake images, all generators, train split
└─ val/
    ├─ 0_real/  ← real images, held-out val portion
    └─ 1_fake/  ← fake images, held-out val portion
```

Labels are inferred from folder name: 0_real = label 0, 1_fake = label 1.

Evaluation

After training, evaluate the saved model on genimage_test/ for each generator separately. For each generator, compute:

- AP : Average Precision (sklearn.metrics.average_precision_score)

- Accuracy : at threshold 0.5
- AUC : Area Under ROC Curve (sklearn.metrics.roc_auc_score)
- Std Deviation — Standard Deviation of AP across all generators. Smaller value means more consistent generalization across different AI generators.

Generator	Type	AP	Accuracy	AUC
Midjourney	Diffusion			
SD v1.4	Diffusion			
SD v1.5	Diffusion			
Wukong	Diffusion			
VQDM	Diffusion			
ADM	Diffusion			
GLIDE	Diffusion			
BigGAN	GAN			
Overall Average				
GAN Average				
Diffusion Average				
Std Deviation				

Approach 1 : RGB + Frequency Masking

What does: fine-tunes the pretrained FakeImageDetection model on GenImage. The model receives standard RGB images. Frequency Masking is kept active during training to preserve the model's generalization capability. Early layers remain frozen throughout all training.

Repository: <https://github.com/chandlerbing65nm/FakeImageDetection>

The repository provides two sets of checkpoints — without Fourier Masking (mask_0) and with Fourier Masking (mask_15). Each set contains three backbone variants:

Checkpoint Folder	File	Backbone	Model Name Arg	Use for Approach 1?
checkpoints/mask_0/	rn50ft.pth	ResNet50 — networks/resnet.py	RN50	no Fourier Masking
checkpoints/mask_0/	rn50_modft.pth	ResNet50 Mod — networks/resnet_mod.py	RN50_mod	No
checkpoints/mask_0/	clipft.pth	CLIP ViT	CLIP_vitl14	wrong architecture
checkpoints/mask_15/	rn50ft_fouriermask.pth	ResNet50 — networks/resnet.py	RN50	✓ USE THIS ONE
checkpoints/mask_15/	rn50_modft_fouriermask.pth	ResNet50 Mod — networks/resnet_mod.py	RN50_mod	different architecture
checkpoints/mask_15/	clipft_fouriermask.pth	CLIP ViT	CLIP_vitl14	wrong architecture

Implementation steps:

1. **Download** the weights from the link provided in the FakeImageDetection README. Place the file at exactly this path inside the cloned repository:

`FakeImageDetection/checkpoints/mask_15/rn50ft_fouriermask.pth`

Do not rename the file. The training script references it by this path.

File	Purpose	Action
train.py	Main distributed training script	Modify: data path, freezing, fc (change_output), save logic
train.sh	Bash launcher for train.py	Modify: set MODEL_NAME=RN50, MASK_TYPE=fourier, RATIO=15, BAND=all
test.py	Evaluation script — AP, Accuracy, AUC	Modify: data path for genimage_test/
test.sh	Bash launcher for test.py	Modify: set MODEL_NAME=RN50, MASK_TYPE=fourier
networks/resnet.py	ResNet50 architecture (RN50 variant — Wang et al.)	Use as-is — fc replaced via change_output(1) in train.py
augment.py	Augmentation logic including Fourier masking	Use as-is , do not modify
mask.py	Masking implementation (FFT, band selection)	Use as-is , do not modify
dataset.py	Dataset loading and transforms	Modify: data paths for GenImage
earlystop.py	Early stopping logic	Use as-is

environment.yml	Conda environment , lists all required packages	Use to install dependencies: conda env create -f environment.yml
checkpoints/mask_15/rn50ft_fouriermask.pth	Pretrained weights , RN50 + Fourier Masking	Download from Google Drive link in README

****Hardcoded paths:** train.py, test.py, and dataset.py contain hardcoded paths under /mnt/SCRATCH/chadolor/Datasets/... update these paths to match the local environment before running anything. The README explicitly warns about this.

2. **Preprocessing** step: Apply to every image at every stage — training, validation, and testing.

Step	Action	Value
1	Resize	224×224 pixels, bilinear interpolation
2	ToTensor	Convert PIL image to PyTorch tensor, values [0,1]
3	Normalize	mean=[0.485, 0.456, 0.406] std=[0.229, 0.224, 0.225]

3. Frequency Masking:

- The Frequency Masking logic is already implemented in the repository
- It is applied during training only, never during validation or testing
- It applies to 50% of training images, randomly selecting a frequency band each time
- The mask_ratio is set to 0.15 , 15% of the selected band's coefficients are zeroed out
- Use the existing implementation code
- **What changes:**
 - Point the data path to GenImage instead of the original ProGAN dataset
 - Freeze conv1, bn1, layer1, layer2, set requires_grad=False for these layers
 - Replace the final fc layer by calling model.change_output(1) after loading pretrained weights
 - Pass only trainable parameters (layer3, layer4, fc) to the optimizer
 - Save the best model as forensic_rgb_model.pth based on val AP.
 - Script arguments: Use --model_name RN50, --mask_type fourier, --ratio 15, --band all when running train.py or test.py.
 - Distributed training: The training uses torch.distributed.launch. Launch via: python -m torch.distributed.launch --nproc_per_node=GPU_NUM train.py --[options]. Or use train.sh with GPU IDs: bash train.sh "0" for single GPU.
 - Replace fc using model.change_output(1) in train.py. Do NOT manually create nn.Linear. The model has a built-in method for this.

4. Model Architecture

Base Model: ResNet50 from FakeImageDetection repository (networks/resnet.py)

Load checkpoints/mask_15/rn50ft_fouriermask.pth with strict=False. Then call model.change_output(1) to replace fc for binary classification.

Layer	Action	Notes
conv1	Unchanged — 3 input channels	No re-initialization needed
layer1	Unchanged	Frozen throughout all training
layer2	Unchanged	Frozen throughout all training
layer3	Unchanged	Trainable throughout all training
layer4	Unchanged	Trainable throughout all training
fc	Call model.change_output(1)	Binary output via nn.Linear(2048,1), called after loading weights. Use model.change_output(1) — do NOT manually create nn.Linear

5. Training

Parameter	
Loss Function	BCEWithLogitsLoss
Optimizer	AdamW
Weight Decay	1e-4
Initial Learning Rate	1e-4
LR Scheduler	ReduceLROnPlateau mode=max, factor=0.5, patience=3, monitors val AP
Batch Size	64
Max Epochs	30
Early Stopping Patience	5 epochs
Frequency Masking	On throughout all training, 50% probability, mask_ratio=0.15
num_workers	4
pin_memory	True

1. parameter

2. Freezing layers

Pass only trainable parameters (layer3, layer4, fc) to the AdamW optimizer.

Frozen layers receive no gradient updates at any point.

Layer	Status — ALL Epochs	Reason
conv1	FROZEN requires_grad=False	Preserves core frequency features from ProGAN training
bn1	FROZEN requires_grad=False	Coupled with conv1
layer1	FROZEN requires_grad=False	Preserves low-level frequency-aware representations
layer2	FROZEN requires_grad=False	Preserves mid-level frequency-aware representations
layer3	TRAINABLE requires_grad=True	Adapts to GenImage diverse generators
layer4	TRAINABLE requires_grad=True	Learns GAN vs Diffusion discrimination
fc	TRAINABLE requires_grad=True	Binary classifier for GenImage labels

3. Training Loop

Run until early stopping fires. No phase transition at any epoch.

Each epoch : Training phase:

1. Set model to train mode
2. Iterate training DataLoader, batch size 64
3. For each batch: load image tensors and labels
4. Frequency Masking applies automatically via the existing repository augmentation code (50% of images, training only)
5. Forward pass through model
6. Compute BCEWithLogitsLoss between output and label
7. Zero gradients
8. Backward pass
9. Optimizer step (updates layer3, layer4, fc only)
10. Accumulate batch loss

Each epoch: Validation phase:

11. Set model to eval mode
12. Iterate validation DataLoader with torch.no_grad()
13. Apply standard preprocessing only, no Frequency Masking
14. Forward pass, apply sigmoid to outputs
15. Collect all predictions and labels
16. Compute AP and Accuracy
17. Print: epoch number, average training loss, val AP, val accuracy
18. Step ReduceLROnPlateau scheduler with current val AP
19. If val AP > best_AP: save model to forensic_rgb_model.pth, update best_AP, reset patience counter
20. Else: increment patience counter
21. If patience >= 5: stop training

6. Evaluation

Load forensic_rgb_model.pth. Set model to eval mode.

For each generator folder in genimage_test/:

- Load images with standard preprocessing (Resize + Normalize) without Frequency Masking
- Run inference with torch.no_grad()
- Apply sigmoid to model outputs
- Compute AP, Accuracy, AUC

- Record results in the evaluation table

Fill the evaluation table

Approach 2: DCT Frequency Domain

What does: Converts every image to a DCT frequency feature map before training.

Output files: forensic_dct_model.pth and dct_stats.json

Implementation steps:

1. DCT Preprocessing

- Precomputed once before training. Saved as .pt files. Applied identically at training, validation, and testing.
- Resize every image to exactly 224×224 pixels using bilinear interpolation.
- Convert 224×224 RGB to YCbCr. Extract Y channel (index 0). Discard Cb and Cr. Convert to float32. Output shape: [224, 224], values in [0, 255].
- **Patch Division:** Process Set A and Set B independently through Steps a and b.
 - a. Apply 2D DCT Type-II to each patch independently. Use `scipy.fftpack.dct`, `norm='ortho'`, along `axis=0` then `axis=1`. Patch-level only.
 - b. Log Scaling: $\log(|\text{coefficient}| + 1e-8)$ applied element-wise to every DCT coefficient.
- Set A (784 patches of 8×8) → reassemble to [224, 224]. Set B (196 patches of 16×16) → reassemble to [224, 224]. Each patch returns to its original spatial position in the grid.

Set	Patch Size	Number of Patches	Stride
Set A	8×8	28×28 = 784	8 (no overlap)
Set B	16×16	14×14 = 196	16 (no overlap)

- Compute element-wise average of the two [224, 224] maps. Add channel dimension. Final shape: [1, 224, 224].
- **Z-Score Normalization:** Iterate all images in `genimage_train/train/` through all previous steps.
 - a. Compute `DCT_mean` and `DCT_std` as scalar values. Save both to `dct_stats.json`. These two numbers are fixed for the entire project.
 - b. **Apply at every stage** (Training, Validation, testing, Any new inference) **using `dct_stats.json`**: Formula: $\text{normalized} = (\text{tensor} - \text{DCT_mean}) / (\text{DCT_std} + 1e-8)$.
 - c. **Do not** recompute `DCT_mean` or `DCT_std` from val or test data.
- Save each tensor as a PyTorch .pt file. Shape: [1, 224, 224]. dtype: float32.

2. Preprocessing Execution Order (Before Training)

- a. Compute `DCT_mean` and `DCT_std` from `genimage_train/train/` → save to `dct_stats.json`

- b. Precompute DCT for all images in genimage_train/train/ → save .pt files
- c. Precompute DCT for all images in genimage_train/val/ → save .pt files
- d. Precompute DCT for all images in genimage_test/[each generator]/ → save .pt files

Steps 2–4 all use the same dct_stats.json values. Run once only — never recompute during training.

3. Model Architecture

Base: Standard torchvision ResNet50 with ImageNet pretrained weights (torchvision.models.resnet50(weights='IMAGENET1K_V1')). Do NOT use any ResNet file from the FakeImageDetection repository.

Modification	Details
conv1 in_channels	Changed from 3 to 1
conv1 out_channels	64 unchanged
conv1 kernel_size	7 unchanged
conv1 stride	2, unchanged
conv1 padding	3, unchanged
conv1 bias	False, unchanged
conv1 weight init	Kaiming Normal, mode='fan_out', nonlinearity='relu'
Load ImageNet weights	strict=False, conv1 skipped, all other layers loaded normally
fc	Replaced with Linear(2048, 1), no activation on output

4. Training

a. Parameters

Parameter	Phase 1 Value	Phase 2 Value
Epochs	1 to 5	6 to max (or early stop)
Learning Rate	1e-4	1e-5
Loss Function	BCEWithLogitsLoss	BCEWithLogitsLoss
Optimizer	AdamW, trainable params only	AdamW, ALL params, reinitialize
Weight Decay	1e-4	1e-4
LR Scheduler	ReduceLROnPlateau mode=max	Reinitialize with new optimizer
Batch Size	64	64
Gradient Clipping	OFF	max_norm=1.0 every training step
layer1 status	FROZEN	TRAINABLE , unfrozen
layer2 status	FROZEN	TRAINABLE , unfrozen
Early Stopping	patience=5 based on val AP	continues from Phase 1
num_workers	4	4
pin_memory	True	True

b. Strategy

Phase 1 — Epochs 1 to 5:

Layer	Status
conv1	TRAINABLE
bn1	TRAINABLE

layer1	FROZEN
layer2	FROZEN
layer3	TRAINABLE
layer4	TRAINABLE
fc	TRAINABLE

Phase 2 — Epoch 6 :

Layer	Status
conv1	TRAINABLE
bn1	TRAINABLE
layer1	TRAINABLE ,NOW UNFROZEN
layer2	TRAINABLE , NOW UNFROZEN
layer3	TRAINABLE
layer4	TRAINABLE
fc	TRAINABLE

5. Training Loop

Phase 1 — Epochs 1 to 5:

Training phase each epoch:

1. Set model to train mode
2. Iterate training DataLoader, batch size 64
3. For each batch: load precomputed .pt tensors and labels
4. Forward pass through model
5. Compute BCEWithLogitsLoss
6. Zero gradients
7. Backward pass
8. Optimizer step — updates conv1, bn1, layer3, layer4, fc only
9. Accumulate batch loss

Validation phase each epoch:

10. Set model to eval mode, torch.no_grad()
11. Iterate validation DataLoader, load precomputed .pt tensors
12. Forward pass, apply sigmoid to outputs
13. Collect all predictions and labels
14. Compute AP and Accuracy
15. Print: epoch, training loss, val AP, val accuracy
16. Step ReduceLROnPlateau with val AP
17. If val AP > best_AP: save model to forensic_dct_model.pth, update best_AP, reset patience
18. Else: increment patience counter

19. If patience ≥ 5 : stop training early

Phase 2 Transition: Execute once at the START of epoch 6:

20. Set requires_grad=True for layer1 and layer2
21. Re-initialize AdamW optimizer with ALL parameters at LR=1e-5, weight_decay=1e-4
22. Re-initialize ReduceLROnPlateau scheduler with the new optimizer
23. Enable gradient clipping: apply torch.nn.utils.clip_grad_norm_max_norm=1.0 every batch

Phase 2 : Epoch 6 onwards:

Training phase each epoch (same as Phase 1 but with gradient clipping added):

24. Set model to train mode
25. Iterate training DataLoader, batch size 64
26. Load precomputed .pt tensors and labels
27. Forward pass through model
28. Compute BCEWithLogitsLoss
29. Zero gradients
30. Backward pass
31. Apply gradient clipping: torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
32. Optimizer step : updates ALL layers including layer1 and layer2
33. Accumulate batch loss

Validation phase each epoch: identical to Phase 1 validation.

6. Evaluation

Load forensic_dct_model.pth. Set model to eval mode.

For each generator folder in genimage_test/:

- Load precomputed .pt files (normalized using same dct_stats.json as training)
- Run inference with torch.no_grad()
- Apply sigmoid to model outputs
- Compute AP, Accuracy, AUC
- Record results in the evaluation table